DÉFI ENEDIS · COMPUTE ENGINE

The dispatch model: decision record

Which model computes the plan, why this one, and the evidence behind it

*Version 1 · 10 October 2026 · Status: **proposed for team adoption**. Everything else (LLM, interface, learning loops) plugs into the contract defined here.*

The decision in one box
We fix one explicit model and one solver around it: RSLS, the Rolling-horizon Smith + Local-Search dispatcher.

1. Model. Crews are parallel machines. Each incident has a weight w = customers × (1 + β·sensitive), a release time, a repair time and a travel time that depends on where the crew comes from. We minimise Σ w·(C − r) (weighted outage hours), plus a stability term and an optional starvation guard. Hard rules: one job at a time, no interruption, 12 h work / 11 h rest, locks and operator rules.
2. Representation. A plan is one ordered list of incidents per crew. A deterministic **decoder** turns the lists into times and inserts rests. Every plan it outputs is feasible by construction.
3. Solver. Smith-rule construction (w / (travel + repair)) in milliseconds, then iterated local search (relocate, swap, reorder, ruin & recreate) within a 200–500 ms budget, then robust selection over duration scenarios.
4. Operation. Re-plan at every event from the previous plan. Commit only the next job of each idle crew. Publish a new plan only if it is clearly better.
5. Explanations come from the model, not from the LLM: priority index, cost breakdown, counterfactual Δ, binding constraint, tipping point, uncertainty band.

Evidence (prototype on the real data, official indicator Σ N·T)	Result
Smith rule with travel vs the 4 other construction rules, 100 crises	best on average; −13.9% vs first-come-first-served, −5.3% vs nearest
Full engine (re-plan at each decision + 50 ms local search, **no knowledge of the future**) vs Smith rule alone, 20 crises	**−14.6%** (range −7.2% to −20.8%), better on 20/20
Full engine vs best simple baseline (most customers first), 20 crises	**−18.9%**, better on 20/20
Full engine vs first come first served / nearest, 20 crises	**−27.6%** / **−20.1%**, better on 20/20
Share of the hindsight headroom captured online	14.6% of the 17.7% achievable with perfect foresight (≈ 83%)

1. The problem as a formal model

This is the contract every component relies on. Every symbol below is a parameter or a data field. None is hard-coded.

1.1 Sets and data

Symbol	Meaning	Source
I, K	incidents known so far; crews (teams)	event stream; agents.csv
r_i	time incident i is reported (release)	dateHeureIncident
p_i	estimated repair time (2, 4 or 8 h); the truth is revealed when the repair ends	dureeDepannage
n_i, s_i	customers cut off; sensitive flag (0/1)	poste.csv
$w_i = n_i(1 + \beta\, s_i)$	weight. Official mode β = 0 (the jury's N×T); operational mode β > 0	config
τ(a, b)	travel time = straight-line distance × 1.3 / speed (replaceable by a road matrix)	config / modifiers
W = 12 h, R = 11 h	work window length, rest length	config
a_k, e_k, o_k	time crew k becomes free, end of its current window, its position	state

1.2 Decision

For each crew k, an ordered list π_k of incidents. Start and completion times C_i are **not** decision variables: the decoder (section 2) computes them from the lists. That keeps the search space purely combinatorial and guarantees feasibility.

1.3 Objective

```
minimise  J = Σ_i f_i(C_i)  +  λ_stab · (number of crews whose next job changes vs the published plan)

          f_i(C_i) = w_i · (C_i − r_i)                          weighted outage hours
                   + γ · n_i · max(0, C_i − r_i − D)             starvation guard (optional)
```

Mode	Setting	Use
Official	β = 0, γ = 0, λ_{stab} = 0	exactly Σ $N_i \cdot T_i$ from the README; used for every benchmark
Operational	β ≈ 3, γ > 0 (D ≈ 24 h), λ_{stab} > 0	what the manager runs; every weight visible and adjustable

Why this exact shape
Each f_i is **non-decreasing** in its own completion time, and J is a **sum over incidents**. That gives three properties we rely on. (1) Finishing a job earlier never hurts, so the decoder can schedule as early as possible. (2) A move in local search changes only the costs of the one or two crews it touches, so evaluation is cheap. (3) The score splits per incident and per term, which is what the explanations are made of. That is why the fairness guard is a per-incident soft deadline and not a non-separable 'max wait' term.

1.4 Constraints and where each one is enforced

#	Rule	Enforced by
C1	each incident handled by q_i crews (today q = 1, parameter)	representation (an incident is in exactly one list)
C2	one job at a time, jobs in sequence with travel in between	decoder
C3	no anticipation: a crew cannot leave for an incident that is not reported yet	state (only known incidents are planned)
C4	no interruption of a repair	decoder

#	Rule	Enforced by
C5	travel + repair must fit entirely in the current 12 h window; otherwise 11 h rest first	decoder
C6	rest on site (or at base, parameter)	decoder
C7	jobs in progress are locked; operator force / forbid rules	move filter (locked items never move)
C8	crew unavailability windows (stress tests, operator)	decoder (window removed)
C9	optional hard deadline for sensitive customers	decoder check → infeasible move rejected

1.5 Problem class

In scheduling notation: parallel machines with release dates, sequence-dependent set-up times (travel) and availability windows, minimising weighted flow time. Even one machine with release dates is NP-hard, and decisions are taken **online** as incidents arrive. So we need a fast heuristic with a sound basis, an exact solver for reference only, and re-planning at every event. In the routing literature this is the *multi-crew minimum weighted latency problem* (Dey & Ornik).

2. Representation and decoder

The decoder is the heart of the model. Given one crew's ordered list, it returns the exact timetable and its cost, applying the work rules. Every solver, every baseline and every explanation goes through it, so the rules live in exactly one place.

```
decode(crew k, list π):
  t ← a_k ; end ← e_k ; here ← o_k ; cost ← 0
  for i in π:
      d ← τ(here, i)
      t_go ← max(t, r_i, accessible_from_i)            # never before the incident exists or is reachable
      if t_go + d + p_i > end:                         # C5: does not fit in this window
          t ← t + R ; end ← t + W ; t_go ← max(t, r_i)   # rest 11 h starting now, new 12 h window
          if t_go + d + p_i > end: return INFEASIBLE    # job longer than a whole window
      C_i ← t_go + d + p_i ; cost += f_i(C_i)
      t ← C_i ; here ← i
  return cost, timetable
```

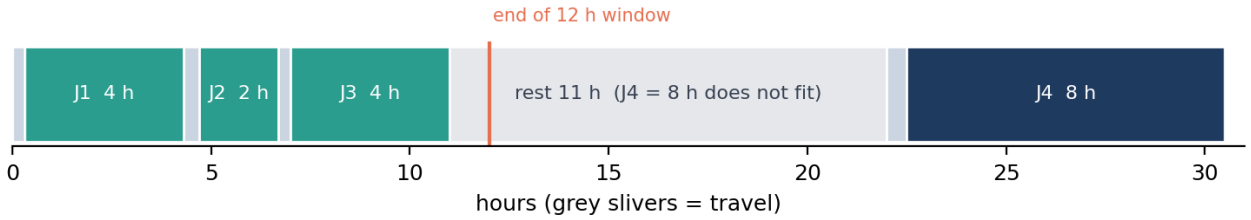


Figure 1. Decoder rule on one crew: work while the next job fits in the window, otherwise rest 11 h first. Here J4 (8 h) cannot start after J3, so the rest is inserted before it.

Properties

- **Feasible by construction.** No solver can produce a plan that breaks C2, C4, C5, C6 or C8, because the decoder is the only way to get times. A separate validator re-checks every published plan anyway.
- **Optimal timing for a given order.** Because each f_i is non-decreasing, starting every job as early as the rules allow is optimal for a fixed order when the crew is never idle. In real time this is the normal case: we only plan incidents that are already reported. The search then only has to find good **orders and assignments**.
- **Exception: idle time.** If a crew sits idle for a long time (no work in its area), resting *during* that idle period can beat working until the window ends. That is a decision about **when crews rest or are mobilised**, not about orders. We handle it as a separate lever (section 4.4), not inside the decoder.
- **Fast.** One call costs O(length of the list), about 3 to 6 incidents per crew in our crises.

3. The solver

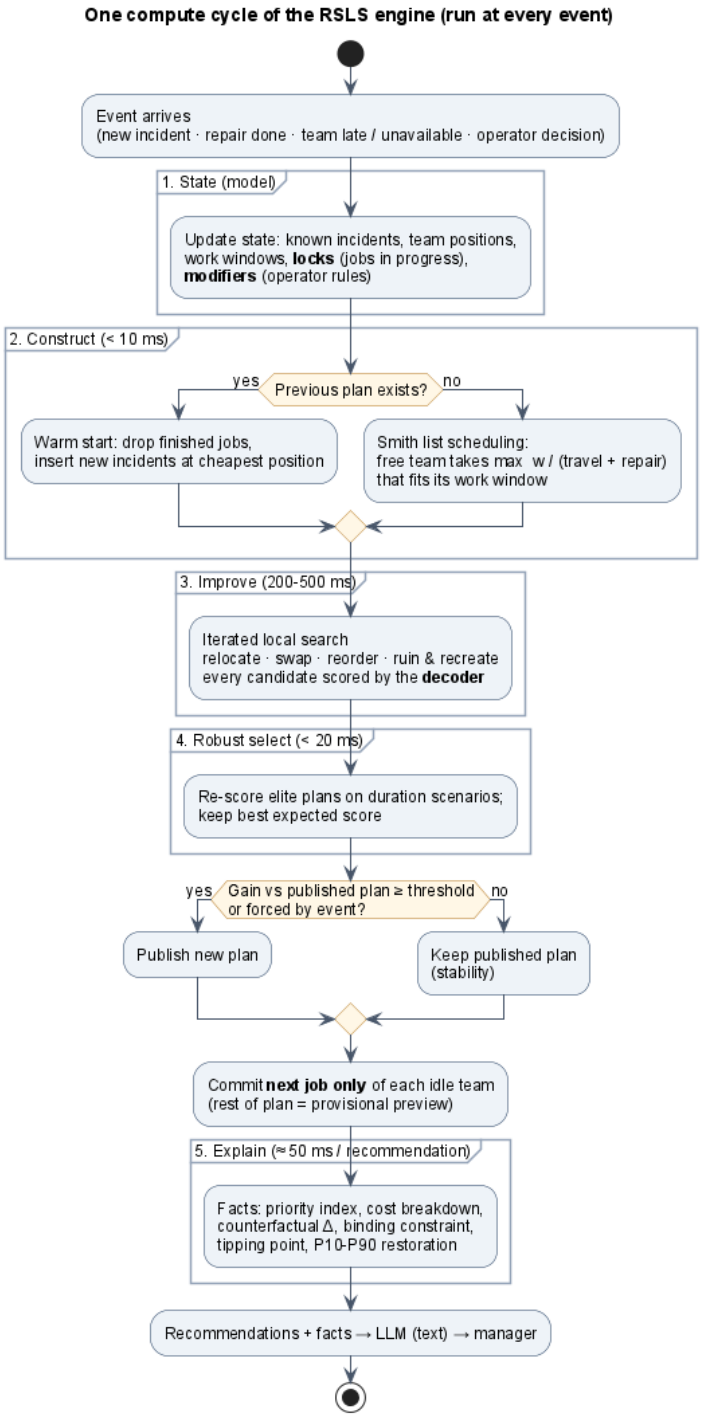


Figure 2. One compute cycle, run at every event. Budgets are per cycle.

3.1 Construction: Smith list scheduling (milliseconds)

```
while unassigned known incidents remain:
    k  ← crew that becomes free first
    i* ← argmax over incidents that fit k's window of   w_i / (τ(k, i) + p_i)
    append i* to π_k ; update k with decode()
    (if nothing fits: k rests 11 h ; if nothing is known: k waits for the next event)
```

Why this rule. w/(τ + p) is *customers restored per hour of crew time*. With one crew and no travel it is exactly Smith's rule, which is optimal (Smith, 1956). On parallel machines with arrivals over time, rules of this family have proven worst-case guarantees (Megow & Schulz 2004; Correa & Wagner 2009). For post-disaster repairs, Tan et al. show an interpretable importance-based dispatch rule with a (2 − 1/m) approximation guarantee. These guarantees hold for simpler models (no travel, no work windows): we say our rule is *grounded in* them, not *guaranteed by* them. **Measured:** it is the best of the five construction rules tested on the 100 crises (Figure 4, left).

3.2 Improvement: iterated local search (200–500 ms)

Component	Choice	Why
Neighbourhoods	relocate (move an incident to another crew / position), swap (exchange two incidents between crews), reorder (within a crew)	the moves that work for minimum-latency problems (transfers and swaps, Dey & Ornik); each touches ≤ 2 crews
Perturbation	ruin & recreate: remove 10–20% of incidents (random, a geographic cluster, or the costliest), reinsert by regret	escapes local optima; the standard approach in modern routing heuristics
Acceptance	improve, or late-acceptance (accept if better than the cost L iterations ago)	one parameter, robust, no temperature tuning
Evaluation	decoder on the 1–2 changed crews only	thousands of moves per 100 ms in Python
Stop	time budget; always return the best feasible plan found	anytime: the construction is already a valid answer
Determinism	seeded random generator	same input, same plan; needed for demos and tests

Measured: a minimal version (relocate moves only, first improvement, 50 ms per decision) already cuts the Smith-rule plan by **14.6%** on average, with **no knowledge of future incidents** (Figure 5). Re-planning *without* the search gives almost nothing. The gain comes from the improvement step, so it is not optional.

3.3 Robust selection under duration uncertainty

Repair times are estimates. The search keeps an elite set of about 5 good plans. Each is re-decoded on S ≈ 30 sampled duration scenarios (lognormal around the estimate), and we keep the best expected score. This costs only milliseconds, because a full decode costs microseconds per crew. The same samples give each recommendation a **P10–P90 restoration band** for the explanation.

3.4 Timing budget per cycle (target)

Step	Budget	Notes
State update + warm start	< 10 ms	insert new incidents at their cheapest position
Local search	200–500 ms	configurable (solver.budget_ms)
Robust selection	< 20 ms	5 plans × 30 scenarios
Explanation facts	≈ 50 ms per new recommendation	counterfactual = short warm-started re-search
Prototype measurement	4.3 s per whole crisis on average	about 160 decisions × 50 ms, pure Python

4. Real-time operation

4.1 Rolling horizon

At every event, the engine plans **all known pending incidents**. At our scale (≤ 250 incidents, 63 crews) no truncation is needed. It starts from the previous plan (warm start), so consecutive plans stay close and the cycle is fast.

4.2 Commit only the next job

Only the **next job of each idle crew** becomes a recommendation the manager validates. The following 2 jobs are shown as a provisional preview. This is the cadence of the damage-assessment allocation system used in production during live storms at Eversource (k jobs per crew, recomputed on a cadence and on operator request).

4.3 Stability: two safeguards

- λ_{stab} in the objective: changing a crew's next job costs a fixed number of customer-hours, so the search only reshuffles when the gain pays for it.
- **Publication threshold**: a new plan replaces the published one only if it is better by at least a set margin (for example 1%), or if an event forces it (crew unavailable, operator rule). Jobs in progress are never moved.
- Each published plan is versioned, so the interface can show what changed and why.

4.4 Lever outside the inner loop: when crews work and rest

Crises last about 36 h in our prototype, so the 11 h rest is the largest single delay. The work rule does not say when the first window starts. We tested holding part of the crews back:

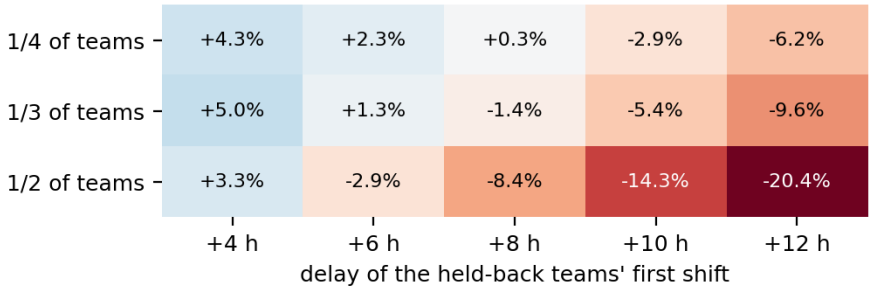


Figure 3. Mean change in customer-hours vs everyone starting at hour 0, 100 crises. A small, short hold-back helps; a large or long one hurts badly. Mobilisation must be decided per crisis, not set by a fixed rule.

So the model has a per-crew **mobilisation time** as an input. It is chosen by an outer loop that simulates a few patterns with the same engine and proposes the best one to the manager. The inner solver stays unchanged. The same outer loop serves pre-positioning and the 'one more crew at site X' lever analysis.

5. Explainability contract

Explanations are computed from the model and handed to the LLM as facts. The LLM writes the text and may not state a number that is not in the facts. This follows the solver-grounded approach recommended in the literature: explanations derived from the optimisation model, not a post-hoc layer (Khan 2026; Čyras et al. 2019). The LLM's role follows OptiGuide (Microsoft) and OptiChat: it translates and calls tools, it never computes.

Fact	How the engine computes it	What the manager reads
Priority index	w / (τ + p) for the chosen pair, and its rank	"brings back 31 customers per hour of work, 2nd highest"
Cost breakdown	f_i split into waiting, travel, repair (customer-hours)	"most of the cost is waiting, not travel"
Contrastive Δ	force the best alternative crew (or incident), short warm-started re-search, compare J	"team 4 instead would cost +410 customer-hours"
Binding constraint	decoder trace (rest inserted, window end, lock, operator rule)	"team 9 cannot: its window ends in 1 h 40"
Tipping point	bisection on p_i (or τ) until the decision flips	"if the repair takes more than 5 h, team 9 becomes better"
Uncertainty band	P10–P90 of C_i over the duration scenarios	"power back between 14:10 and 16:40"
Override cost	re-plan with the operator's choice locked, compare J	"your choice costs +2% (+320 customer-hours)"


```
{"team": "SITE_07_E2", "incident": "INC_004211", "rank": 1,
 "factors": {"customers": 120, "sensitive": false, "travel_min": 25, "repair_h": 4, "window_left_h": 6.1,
             "priority_index": 27.4},
 "alternative_team": {"team": "SITE_13_E1", "delta": 410}, "binding": null,
 "tipping_point": {"param": "repair_h", "value": 5.2, "flips_to": "SITE_15_E3"},
 "restoration_p10_p90": ["14:10", "16:40"]}
```


Example of what the LLM writes from this (illustrative values): « Équipe 2 de Muret → poste de Seysses : 120 clients, 25 min de route, 4 h de dépannage. Envoyer l'équipe de Saint-Gaudens coûterait 410 clients·h de plus. Si le dépannage dépasse 5 h 12, l'équipe 3 de Boussens devient préférable. Rétablissement prévu entre 14 h 10 et 16 h 40. »

6. How the model improves without changing the solver

Every improvement discussed (manager feedback, expert rules, calibration, learned preferences) acts on **data the model reads**, never on solver code. The solver stays one tested piece.

What changes	Where it enters the model	Who changes it
Operator rules (road cut, crew unavailable, force / forbid, priority)	modifiers → decoder inputs or move filters	manager, via LLM + validator
Permanent rules (after repeated refusals)	constraint registry (versioned)	experts, after a replay test
Repair / travel estimates	p_i, τ factors	calibration from observed outcomes
β, γ, D, λ_{stab}, publication threshold	objective weights	manager / learned from chosen vs rejected plans
Index variant w / (p + θτ)	construction rule parameter θ	tuned by replaying the 100 crises
Mobilisation times, pre-positioning	crew windows and positions (inputs)	outer simulation loop, manager approval
Several crews per incident, skills	q_i > 1 in the decoder (start when all arrive); eligibility filter	future data

7. Evidence from the prototype

Setup. Real data (63 crews, 100 crises), straight-line travel × 1.3 at 50 km/h, deterministic durations (estimate = actual), all crews start at hour 0, official objective Σ N·T. The engine sees an incident only once it is reported. Construction rules were run on all 100 crises. The full engine (re-plan at each decision + relocate local search, 50 ms) was run on 20 crises (every 5th one) because Python run time is longer.

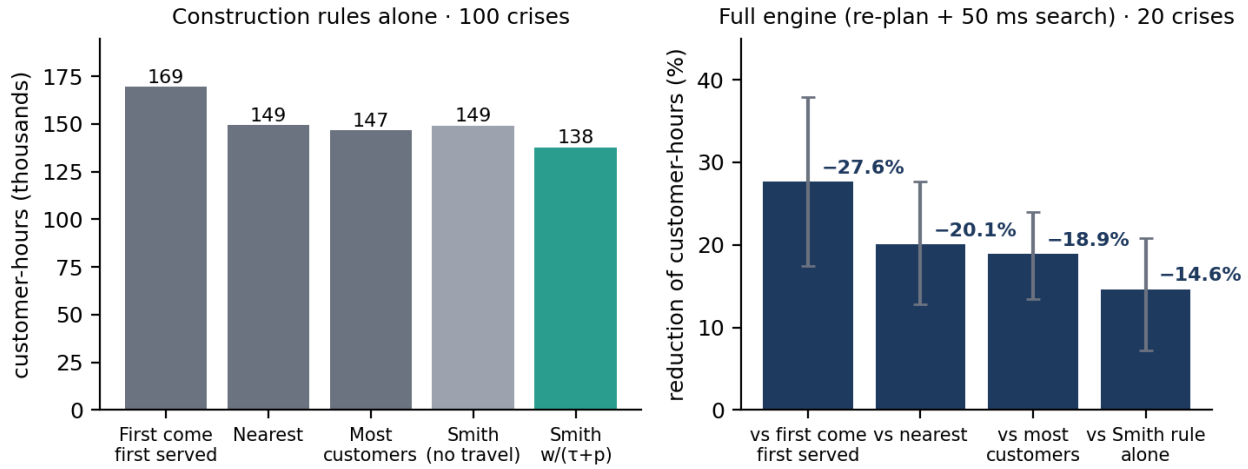


Figure 4. Left: construction rules alone (mean customer-hours per crisis, lower is better). Right: full engine vs each strategy on the same 20 crises (bars = mean, whiskers = min–max).

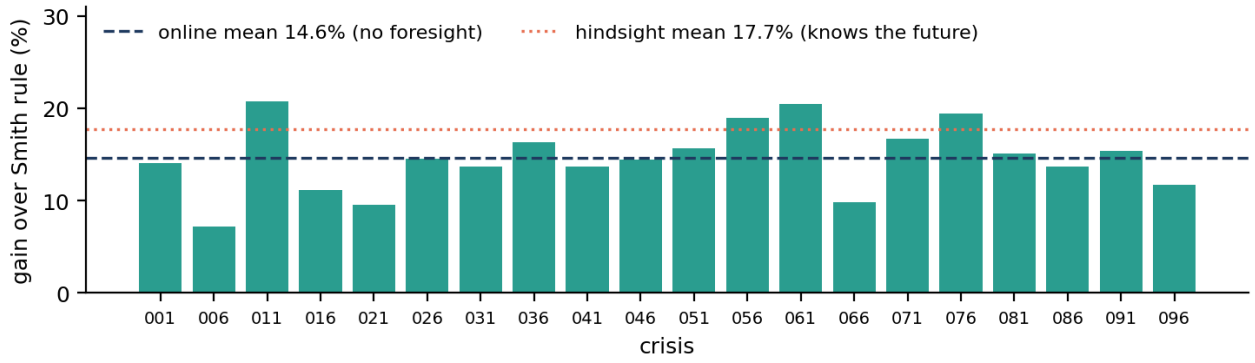


Figure 5. Gain of the full engine over the Smith rule alone, per crisis. Online (no foresight) it captures most of the gain that local search reaches when it knows all incidents in advance.

Comparison (same crises, paired)	Mean	Range	Better on
Full engine vs first come first served	−27.6%	−17.4% … −37.9%	20/20
Full engine vs nearest crew	−20.1%	−12.8% … −27.6%	20/20
Full engine vs most customers first	−18.9%	−13.4% … −24.0%	20/20
Full engine vs Smith rule alone	−14.6%	−7.2% … −20.8%	20/20
Smith rule vs nearest, 100 crises	−5.3%	-14.8% … +7.5%	81/100

What we learn

(1) The construction rule alone is only a few percent better than simple rules, and it loses to 'nearest' on about one crisis in five. **(2)** The local search is what makes the difference: about −15% on top of the rule, every time. **(3)** Re-planning without search adds almost nothing. **(4)** Rest timing is a big lever, but a risky one. **Caveats:** prototype only (relocate moves, 50 ms, no stability penalty, no duration noise, 20-crisis sample for the full engine). The stability penalty will give back part of the gain in exchange for calmer plans. Re-measure on all 100 crises once the real engine exists.

8. Alternatives considered

Option	Verdict	Reason
Exact MILP / CP-SAT as the real-time engine	reference only	travel depends on the previous job, which needs routing (circuit) constraints; seconds to minutes with no guarantee at 250 × 63; optimal on *estimated* data is false precision. Kept offline to measure the heuristic's gap.
Priority rule alone (no search)	rejected as final engine	measured: leaves about 15% on the table
Genetic algorithm	rejected	slower and less controllable than local search at this size, no gain in explainability
Reinforcement learning (AlphaZero-style dispatch)	rejected	needs training, no decision data, opaque, published tests on 8- and 123-node test grids
LLM as planner over a validated action catalogue (LARA)	rejected	conflicts with 'the engine decides, the human validates'; small test grids. We keep its validated-catalogue idea for operator rules.
Pure graph algorithms (matching, flows)	used as parts	exact for one-step assignment or zone balancing, but blind to ordering over time and work windows. Candidates: matching for simultaneous decisions, flows for zone balancing (later).
Off-the-shelf solvers (PyVRP, Timefold)	not adopted	PyVRP optimises route cost, not weighted latency. Timefold Python needs a JVM, is slower than its Java version, and its explanation features are enterprise-side. We borrow its per-constraint score analysis.
OR-Tools Routing	offline reference	the model maps directly (soft cumulative upper bound = Σ w·C, break intervals = rests); used for the quality gap

9. Acceptance criteria for the real engine

Criterion	Target
Feasibility	0 infeasible plans over all 100 crises and every stress test (validator)
Quality	≥ 10% fewer customer-hours than the best simple baseline, mean over the 100 crises, official mode
Gap to reference	measured vs OR-Tools offline (long time limit) on 10 small crises, reported in the README
Latency	p95 of a full compute cycle < 1 s on a laptop at 250 incidents
Stability	mean number of next-job changes per published plan reported; ≤ 2 crews on average
Determinism	same input + same seed → identical plan
Explanations	every recommendation has the facts of section 5; Δ = 0 when forcing the recommended choice

10. Assumptions to confirm with the organisers

- A repair may not cross the end of a 12 h window (we assume no), and travel counts as work time (we assume yes).
- The rest starts as soon as the next job cannot fit (we assume yes); all crews start fresh at hour 0 unless mobilised later.
- After a repair, the crew stays on site (we assume yes); one crew per incident (parameter q).
- The official indicator Σ N·T is to be minimised (README says 'maximiser', read as a typo).
- Repeated incidents on the same substation are separate incidents; the 5 incidents on unknown substations are dropped.

References

- W. E. Smith (1956), Various optimizers for single-stage production, Naval Research Logistics Quarterly.

- N. Megow, A. S. Schulz (2004), On-line scheduling to minimize average completion time revisited, Operations Research Letters 32. J. Correa, M. Wagner (2009), LP-based online scheduling: from single to parallel machines.
- Y. Tan, F. Qiu, A. K. Das, D. S. Kirschen, P. Arabshahi, J. Wang, Scheduling Post-Disaster Repairs in Electricity Distribution Networks, arXiv:1702.08382.
- A. Dey, M. Ornik (2022), Post-Disaster Repair Crew Assignment Optimization Using Minimum Latency, arXiv:2206.00597 (code: github.com/leadcatlab/MWLP-Storm-Repair).
- Optimized Rolling Allocation of Outages for Damage Assessment (Eversource Energy), arXiv:2512.20719.
- B. Li, K. Mellou, B. Zhang, J. Pathuri, I. Menache (2023), Large Language Models for Supply Chain Optimization (OptiGuide), arXiv:2307.03875. H. Chen et al. (2025), OptiChat, INFORMS Journal on Data Science.
- K. Čyras et al. (2019), Argumentation for Explainable Scheduling, AAAI. Khan (2026), solver-grounded explanations for scheduling, arXiv:2603.00469 (preprint).
- Walsh et al., Agent Based Model to Estimate Time to Restoration of Storm-Induced Power Outages, Infrastructures 3(3).
- Note: several references were read as abstracts only. Read the full texts before quoting figures from them in the pitch.